

ELEC/COEN 390

Engineering team design project

MD NAZMUL HOQ

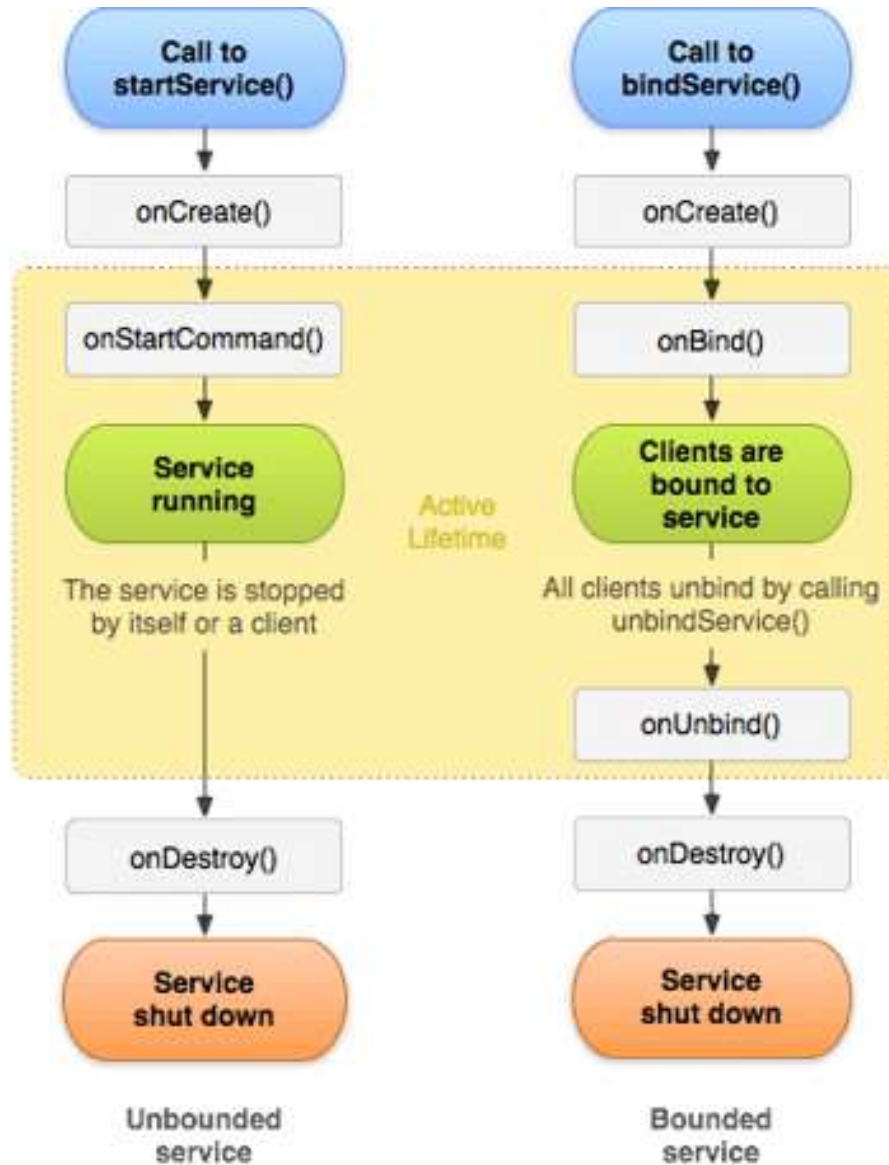
KHADIZA SARWAR MOURY

CONCORDIA UNIVERSITY

Outline

2

- ▶ Service and Foreground Service
- ▶ Java concurrency (multi-threading)
- ▶ Graph Plotting using GraphView
- ▶ Getting Started With Arduino
- ▶ Writing Arduino Programs
- ▶ Flashing firmware to the Arduino
- ▶ Writing Your First Program
- ▶ Other Features of the Arduino Platform
- ▶ Internet Communication
- ▶ Using Volley to retrieve data
- ▶ Creating Your Own Server
- ▶ Firebase cloud service
- ▶ Prototyping



Services

[HTTPS://WWW.TUTLANE.COM/TUTORIAL/ANDROID/ANDROID-SERVICES-WITH-EXAMPLES](https://www.tutlane.com/tutorial/android/android-services-with-examples)

Services

- ▶ A Service is an application component that **can perform long-running operations** in the background
- ▶ Does not provide a user interface
- ▶ two forms:
 - ▶ Started: A service is "started" when an application component (such as an activity) **starts it by calling `startService()`**. Once started, a service can **run in the background indefinitely**, even if the component that started it is **destroyed**.
 - ▶ For example, it might download or upload a file over the network. When the operation is done, the service should stop itself.
 - ▶ Bound: A service is "bound" when an application component binds to it **by calling `bindService()`**. A bound service offers a client-server interface that allows components to interact with the service, send requests, and get results. A bound **service runs only as long as another application component is bound to it. Multiple components can bind to the service at once**, but when all of them unbind, the service is destroyed.

More on services

- ▶ However, you can declare the service as private, in the manifest file, and block access from other applications. This is discussed more in the section about [Declaring the service in the manifest](#).
- ▶ **Caution:** A service runs in the main thread of its hosting process—the service does **not** create its own thread and does **not** run in a separate process (unless you specify otherwise). This means that, if your service is going to do any CPU intensive work or blocking operations (such as MP3 playback or networking), you should create a new thread within the service to do that work.
- ▶ Source and for learning more:
<https://developer.android.com/guide/components/services.html>

Foreground Service

- ▶ If the service is currently executing code in its `onCreate()`, `onStartCommand()`, or `onDestroy()` methods, then the hosting process will be a foreground process to ensure this code can execute without being killed.
- ▶ A started service can use the `startForeground(int, Notification)` API to put the service in a foreground state, where the system considers it to be something the user is actively aware of and thus not a candidate for killing when low on memory. (It is still theoretically possible for the service to be killed under extreme memory pressure from the current foreground application, but in practice this should not be a concern.)

See it in code:

- ▶ Music player
- ▶ <http://www.truiton.com/2014/10/android-foreground-service-example/>
- ▶ Simpler music player foreground service
- ▶ <http://www.101apps.co.za/articles/services-tutorials-part-2-a-foreground-service.html>
- ▶ A third music player
- ▶ <http://www.tutorialsfac.com/2015/09/simple-android-foreground-service-example/>

And you can always go back to the official documentation:

<https://developer.android.com/reference/android/app/Service.html>

Concurrent execution

- ▶ Java concurrency (multi-threading)
 - ▶ <https://developer.android.com/reference/java/util/concurrent/package-summary>
 - ▶ <https://www.vogella.com/tutorials/JavaConcurrency/article.html>
 - ▶ https://subscription.packtpub.com/book/application_development/9781785883248/1/ch01lvl1sec11/concurrency-in-android
- ▶ Replaces the deprecated AsyncTask
 - ▶ <https://developer.android.com/reference/android/os/AsyncTask.html>

Graph Plotting using GraphView

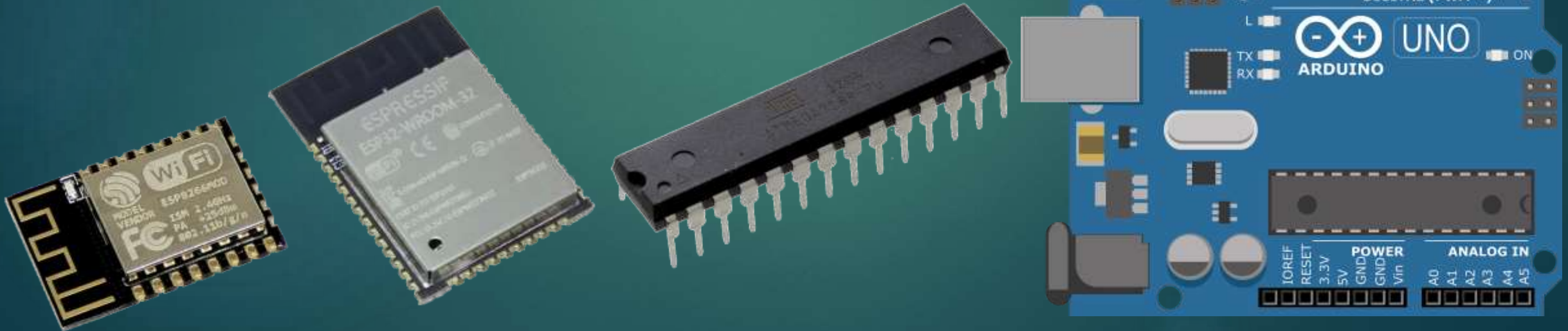
9

- ▶ Your projects will have many interesting features
- ▶ But you have to meet sprint deadlines!
- ▶ **Avoid reinventing the wheel: Use good libraries**
- ▶ Example Feature: Make a graph of altitude
 - ▶ Need to show Altitude over Time in an Activity. How?
 - ▶ Two arrays holding data
 - ▶ Use a Graph Library to display array data!
- ▶ Some useful libraries and resources
 - ▶ <https://github.com/PhilJay/MPAndroidChart#examples>
 - ▶ <http://www.android-graphview.org/>
 - ▶ <https://github.com/JStumpp/awesome-android>

Getting Started With Arduino

10

- ▶ Arduino is an open-source electronics prototyping platform based on flexible, easy-to-use hardware and software.
- ▶ Most Arduino boards are based on Atmel's ATMEGA microcontrollers
- ▶ Espressif's ESP8266/ESP32 are also an option



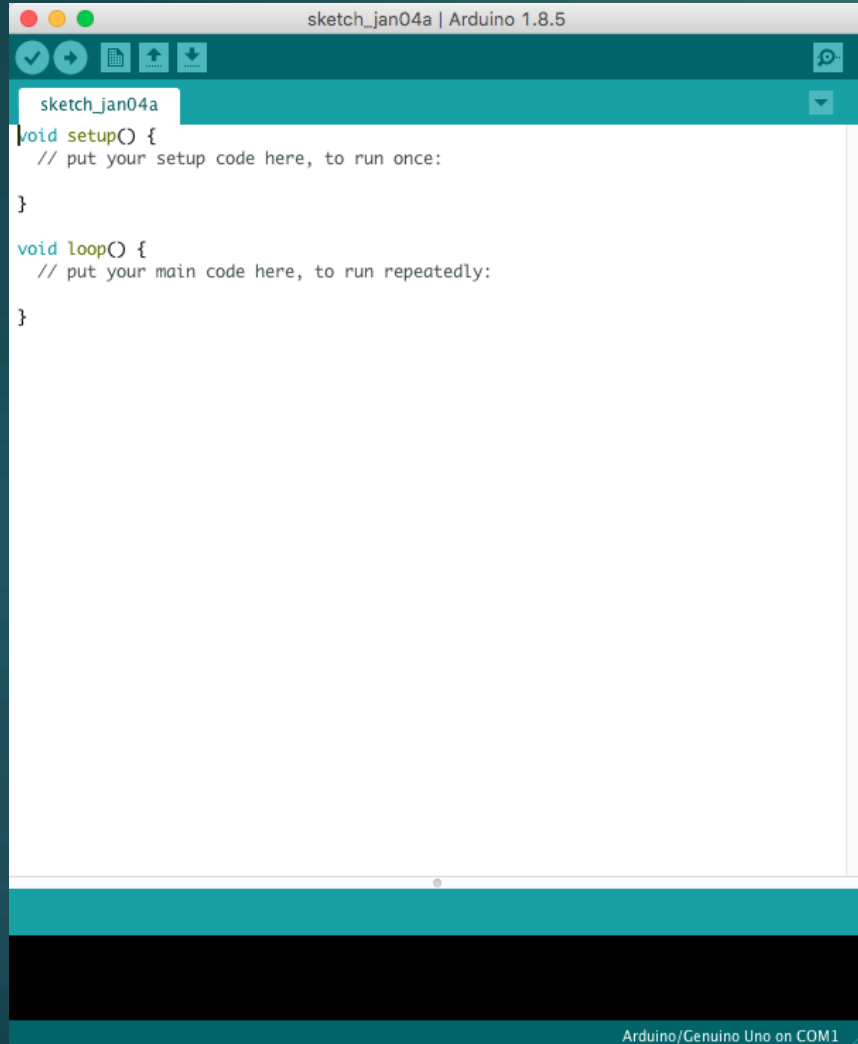
Writing Arduino Programs

11

- ▶ Arduinos can be programmed from a computer with serial communication through a USB cable
- ▶ To program the Arduino boards, there is an Arduino IDE which allows flashing the microcontroller with a program binary
- ▶ The Arduino IDE provides a modified C++ environment to write the program logic
- ▶ Every Arduino file (known as a sketch) contains two functions `setup()` and `loop()`
- ▶ Download the Arduino IDE at:
<https://www.arduino.cc/en/Main/Software>

Arduino IDE

12



- ▶ `setup()` is called **once** when the program is first run
- ▶ `loop()` repeats once it reaches the end indefinitely
- ▶ Notice how there is no `main()` function like all C++ programs usually have

Flashing the Arduino

13

- ▶ Step 1: Write your program
 - ▶ In the main sketch window create your program by writing code for the `setup()` and `loop()` functions
- ▶ Step 2: Select your board
 - ▶ In the toolbar go to Tools > Board > Arduino/Genuino Uno (Select your board here)
- ▶ Step 3: Select the port
 - ▶ In the toolbar go to Tools > Port > `/dev/usb/cu.usbmodem1411` (The port name might be different depending on your operating system)
- ▶ Step 4: Upload the program
 - ▶ In the toolbar go to Sketch > Upload

Writing Your First Program

- ▶ Pin 13 is the built-in LED pin on most Arduinos (check the datasheet to make sure)
- ▶ The program simply turns the built-in LED on and off in a loop

```
int ledPin = 13;

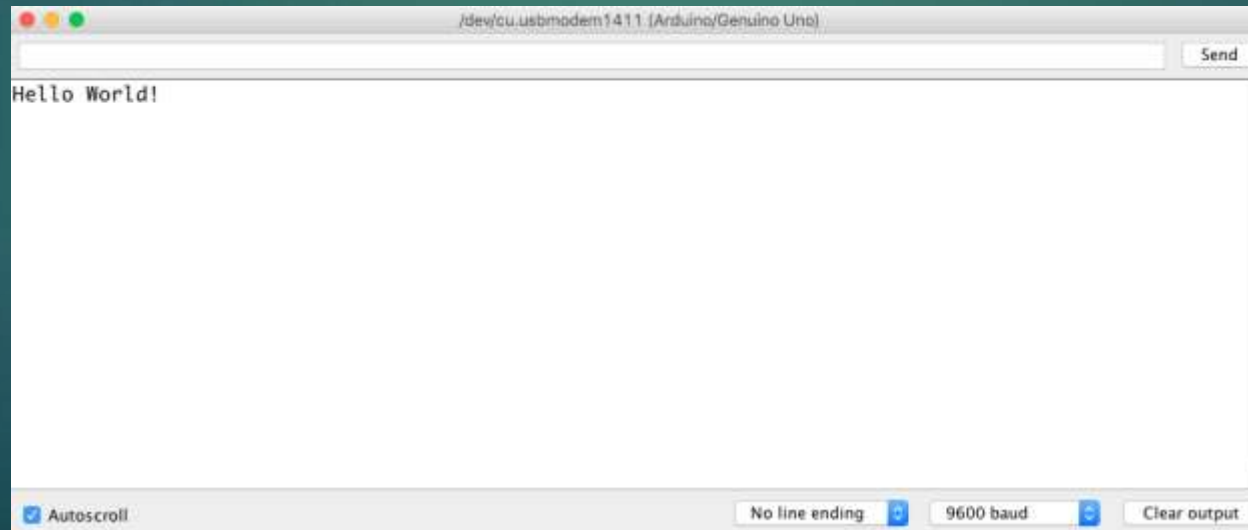
void setup() {
  // initialize digital pin for LED as an output
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // repeatedly turn the led on and off forever
  digitalWrite(ledPin, HIGH);
  delay(1000);
  digitalWrite(ledPin, LOW);
  delay(1000);
}
```

Other Features of Arduino Platform

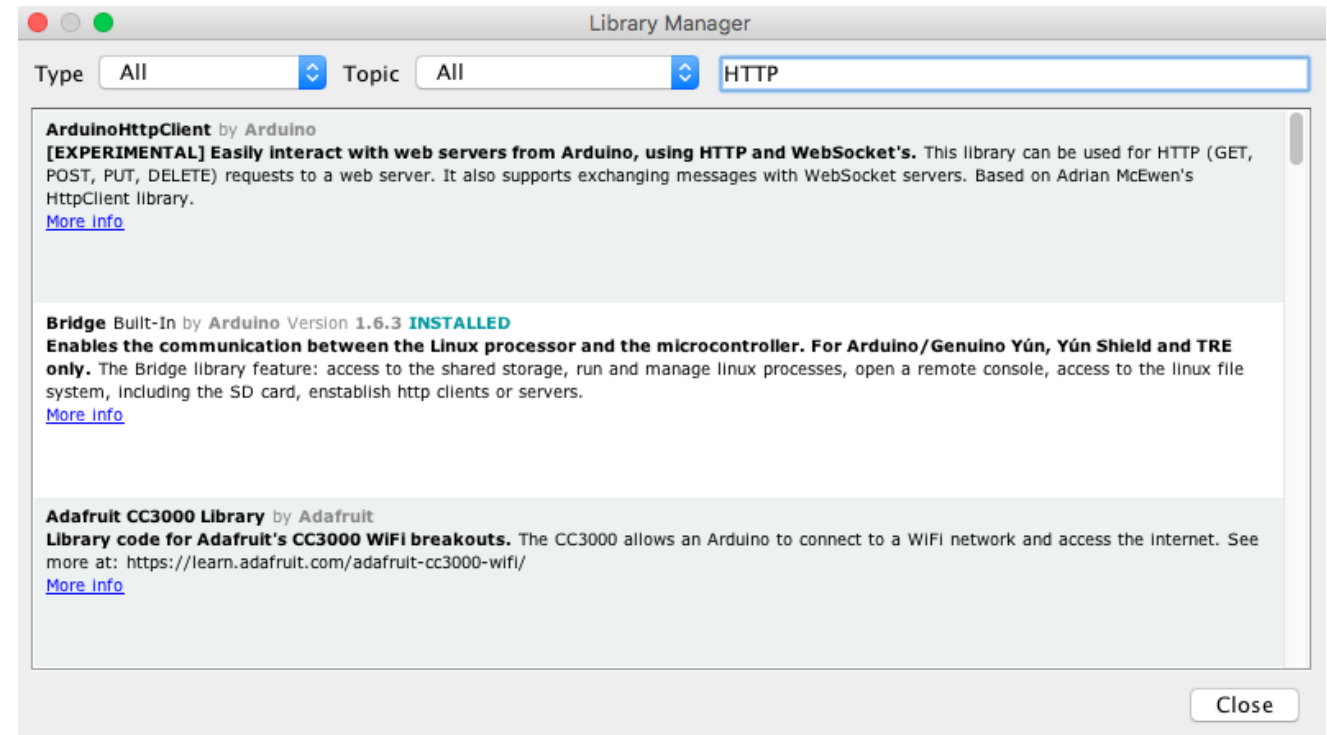
15

- ▶ Useful Serial monitor for serial communication which can be used for printing debug messages
- ▶ Use the function `Serial.begin(9600)` to initialize the serial data transmission where 9600 is the baud rate (data rate in bits/sec)
- ▶ Use the function `Serial.print()` to output your message



Other Features of Arduino Platform

- ▶ The Arduino IDE comes with a easy to use package manager for downloading libraries created by other developers
- ▶ Useful for downloading libraries to access WiFi and Bluetooth libraries
- ▶ To search or download a library in the toolbar go to Sketch > Include Library > Manage Libraries...



Internet Communication

17

- ▶ Communication between an Arduino and Android device is achieved via Internet (on Arduinos with Ethernet or Wi-Fi chips only)
- ▶ The main method of communication for transferring text, images, sound, videos is Hypertext Transfer Protocol (HTTP)
- ▶ A web browser such as Google Chrome is essentially a HTTP client which makes requests to server machines hosting the resources

REST Architecture Style

18

- ▶ REST architecture is used in conjunction with HTTP.
- ▶ REST is a set of constraints which describe how to access a resource
- ▶ The two most common data models for text are: JSON and XML
- ▶ Most modern REST APIs use JSON since it is more lightweight than XML when transferring over the network

```
<?xml version="1.0" encoding="UTF-8" ?>
  <courseName>COEN</courseName>
  <courseNumber>390</courseNumber>
  <assignments>Assignment 1</assignments>
  <assignments>Assignment 2</assignments>
  <teacher>
    <name>William Lynch</name>
  </teacher>
```

Extensible Markup Language (XML)

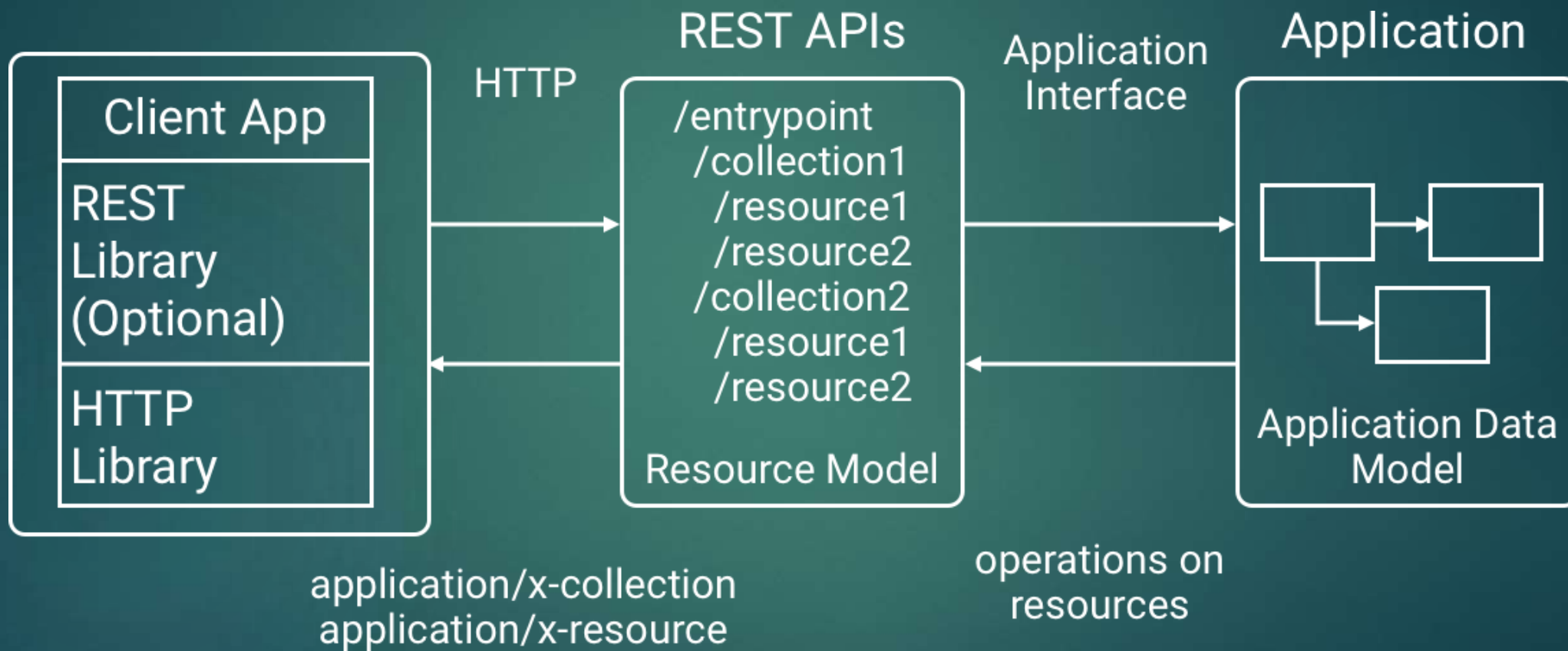
```
{
  "courseName": "COEN",
  "courseNumber": 390,
  "assignments": [
    "Assignment 1",
    "Assignment 2"
  ],
  "teacher": { "name": "William Lynch" }
}
```

JavaScript Object Notation (JSON)

- ▶ A resource is the fundamental concept in any REST API
 - ▶ Resources can be accessed in a variety of formats such as HTML, XML, plain text, PDF, JPEG, JSON and other document formats.
- ▶ Each collection and resource in the API has its own URL
 - ▶ Uniform Resource Locator (URL) is a formatted, case-insensitive string containing the necessary parameters to identify a resource
 - ▶ URI = "http:" "://" host [":" port] [abs_path ["?" query]]

REST API

20



REST Operations

21

- ▶ Methods can be executed on resources via their URL

Method	Semantics
GET	Retrieve a resource
HEAD	Retrieve a resource (header only)
POST	Create a new resource
PUT	Update a resource
PATCH	Update a resource
DELETE	Delete a resource
OPTIONS	Return available HTTP methods and other options

Data Serialization

22

- ▶ Serialization is the process of converting some in-memory object to another format that could be used to either store in a file or sent over the network. Deserialization is the inverse process.
- ▶ JSON and XML need to be serialized (usually to strings) when being sent over the network
- ▶ There are many libraries available which perform the serialization/deserialization process

Data Serialization

23

- ▶ For example: the server responds to the URL `http://mywebsite.com/api/course/1` with:
`{"courseName":"COEN","courseNumber":390,"assignments":["Assignment 1","Assignment 2"],"teacher":{"name":"William Lynch"}}`

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <ArduinoJson.h>

HTTPClient http;
http.begin("http://mywebsite.com/api/course/1 ");

int httpCode = http.GET();
String payload = http.getString();

StaticJsonBuffer<200> jsonBuffer;
JsonObject& root = jsonBuffer.parseObject(payload);

// Retrieve course number from JSON response
// courseNumber should be equal to 390
int courseNumber = root["courseNumber"];
```

Arduino Example using ArduinoJSON

```
import com.android.volley.Request;
import com.android.volley.RequestQueue;
import com.android.volley.Response;
import com.android.volley.toolbox.JsonObjectRequest;
import com.android.volley.toolbox.Volley;
import org.json.JSONObject;

RequestQueue requestQueue;

String url = "http://mywebsite.com/api/course/1";

JsonObjectRequest jsonObjRequest = new JsonObjectRequest(
    Request.Method.GET, url, null, new Response.Listener<JSONObject>() {
        @Override
        public void onResponse(JSONObject response) {
            try {
                int courseNumber = response.get("courseNumber");
            } catch (Exception e) {
                // handle the error
            }
        }
    }
);

requestQueue.add(jsonObjRequest);
```

Android Example using Volley

Android Volley Demo

24

- ▶ Volley is an HTTP library that makes networking for Android apps easier and faster.
- ▶ Volley benefits:
 - ▶ Automatic scheduling of network requests
 - ▶ Multiple concurrent network connections
 - ▶ Support for request prioritization
 - ▶ Debugging and tracing tools
- ▶ Learn more about Volley:
<https://developer.android.com/training/volley/index.html>
- ▶ Simple example, requesting data:
<https://developer.android.com/training/volley/simple#java>

Using Volley

25

- ▶ 1. Add Volley as a dependency in the build.gradle file:

```
dependencies {  
    implementation 'com.android.volley:volley:1.1.0'  
}
```

- ▶ 2. Add Internet permissions in the AndroidManifest.xml file

```
<uses-permission android:name="android.permission.INTERNET"></uses-permission>
```

```

void requestWeather() {
    // Get weather for Montreal
    final String url = "http://api.openweathermap.org/data/2.5/weather?q=Montreal&units=metric&APPID=9208dccec4431655d17d8dfa3d4fab7";

    // Make request
    JsonObjectRequest weatherRequest = new JsonObjectRequest(
        Request.Method.GET, url,
        jsonRequest: null,
        new Response.Listener<JSONObject>() {
            @Override
            public void onResponse(JSONObject response) {
                Log.i(MAIN_LOG_TAG, msg: "response: " + response);
                try {
                    // Get description from weather response
                    String description = response.getJSONArray( name: "weather").getJSONObject( index: 0).getString( name: "main");
                    descriptionTextView.setText(description);

                    // Get temperature from weather response
                    int temperature = response.getJSONObject("main").getInt( name: "temp");
                    temperatureTextView.setText(temperature + "°");

                } catch (Exception e) {
                    // Called when there is an error parsing the json
                }
            }
        },
        new Response.ErrorListener() {
            @Override
            public void onErrorResponse(VolleyError error) {
                // Called when there is an error making the request
            }
        }
    );

    queue.add(weatherRequest);
}

```

Using Volley

3. MAKING THE REQUEST

Creating Your Own Server

27

- ▶ Hosting a server which exposes a REST API can be written in many languages even Java
- ▶ The server simply waits for requests which are made via URLs which you define as part of your REST API
- ▶ There are two options for hosting the server:
 - ▶ Locally: easy to setup, not very portable, hard to collaborate with team
 - ▶ Cloud: difficult to setup for beginners, scalable, easier collaboration
 - ▶ use Amazon Web Services, Azure, Firebase/Google Cloud Engine, Heroku

```
const express = require('express');

const server = express();

server.get('/', (req, res) => {
  res.send('Hello World!');
});

server.get('/api/course/1', (req, res) => {
  res.json({
    courseName: "COEN",
    courseNumber: 390,
    assignments: ["Assignment 1", "Assignment 2"],
    teacher: { name: "William Lynch" }
  });
});

server.listen(3000);
```

▶ Going to localhost:3000/api/course/1 will show the server is running successfully



NodeJS/Express Example

NodeJS/Express Example

29

- ▶ To learn more about create a web server/REST API with NodeJS see:
- ▶ <https://medium.com/@LindaHaviv/the-beginners-guide-understanding-node-js-express-js-fundamentals-e15493462be1>
- ▶ <https://shapedshed.com/creating-a-basic-site-with-node-and-express/>
- ▶ <http://nodeguide.com/beginner.html>
- ▶ <https://nodejs.org/en/>
- ▶ <https://expressjs.com>

```
from flask import Flask
from flask import jsonify

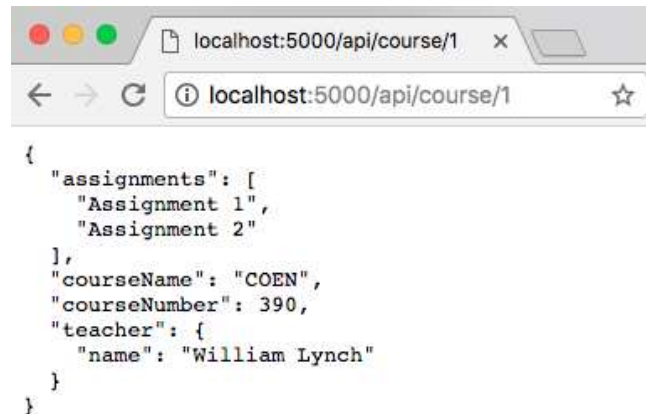
app = Flask(__name__)

@app.route('/')
def index():
    return "Hello World!"

@app.route('/api/course/1')
def course():
    return jsonify({
        "courseName": "COEN",
        "courseNumber": 390,
        "assignments": ["Assignment 1", "Assignment 2"],
        "teacher": { "name": "William Lynch" }
    })

if __name__ == "__main__":
    app.run()
```

Going to localhost:5000/api/course/1 will show the server is running successfully



Python/Flask Example

Python/Flask Example

31

- ▶ To learn more about create a web server/REST API with Python see:
- ▶ <http://blog.luisrei.com/articles/flaskrest.html>
- ▶ <https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-i-hello-world>
- ▶ <http://flask.pocoo.org>
- ▶ <https://www.python.org>

Other Web Server Frameworks

32

- ▶ The same basic principles of a web server and REST API apply to all major frameworks:
- ▶ Java
 - ▶ Spring: <https://spring.io>
 - ▶ Spark: <http://sparkjava.com>
- ▶ Ruby: <http://rubyonrails.org>
- ▶ PHP: <http://php.net>



Realtime Database

- Database is stored under the format of JSON and to be synchronized with clients in real-time.
- Firebase uses NoSQL type for its database

•Offline Work

- The application on Firebase is maintained even the interruption of internet connection. Once the data is written to the cloud, it will be stored in the local database of Firebase. After the internet is reconnected, client's activities will be updated immediately and synchronized the the server promptly.

Support IoT application
with ESP8266/ESP32
boards

+

Mobile SDKs

•User Authentication

- Firebase created the authenticated function by Email, Facebook, Twitter, GitHub, Google, and anonymous. We can integrate this function for the back-end code of the application as tokens or API secret keys.

Good tutorials: IoT sensor + Android + firebase + push notification

<https://www.youtube.com/playlist?list=PLhPDb5zFmGR3qxa8JfmUdAfVH9Ou41MUx>

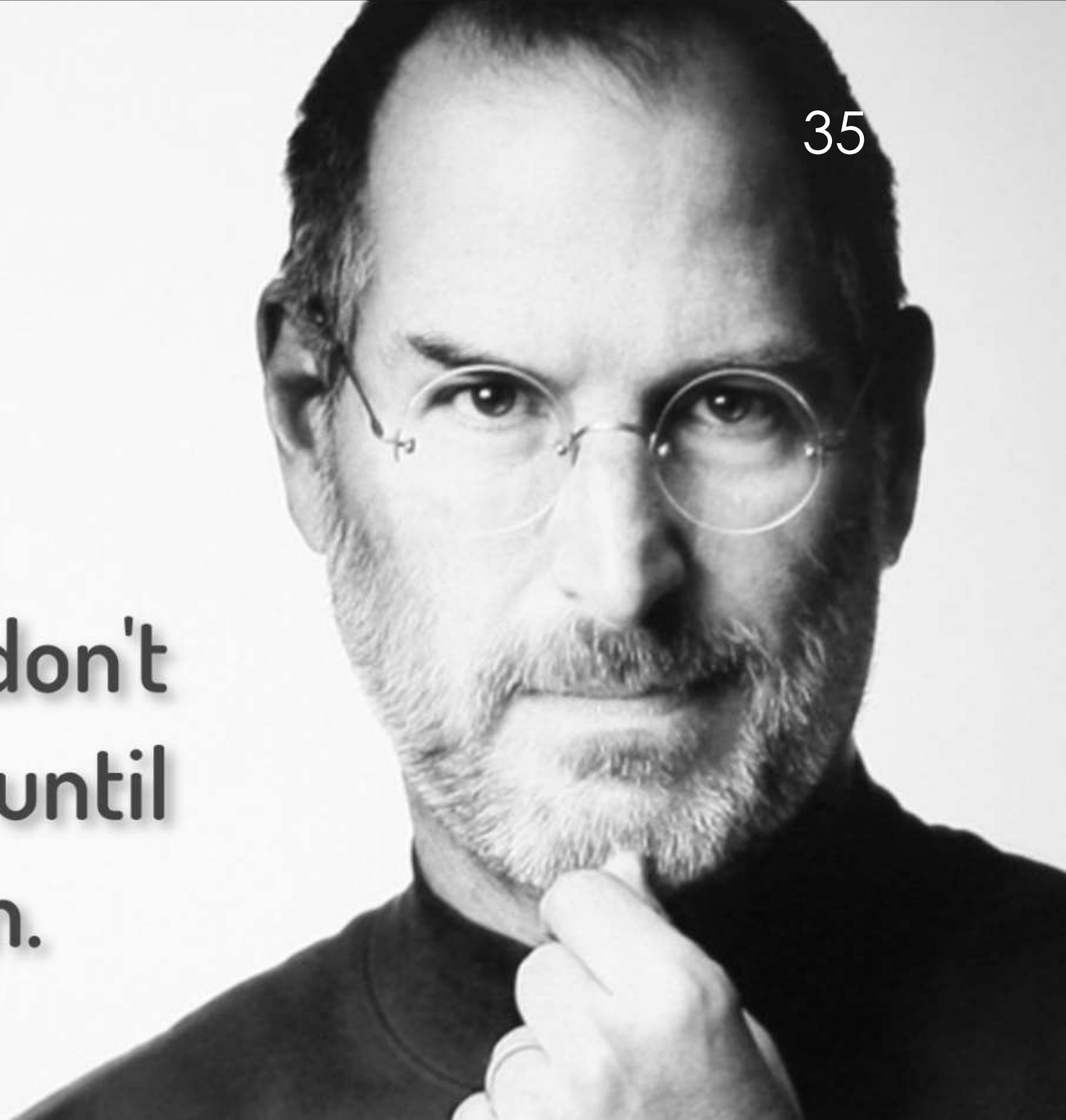
Benefits of Prototyping

34

- ▶ a prototype is not a fully working system, but it gives the developers a better idea of what the final product will look like. and most importantly it provides an opportunity for the customer to give feedback.
- ▶ With the prototyping tools available, you can easily build a prototype of the mobile application that can be simulated on an actual mobile phone.
- ▶ The simulation will not have real data and real functionality but at least it will give the customer a good idea of how the app will look like.

A lot of times, people don't
know what they want until
you show it to them.

- Steve Jobs



PROTOTYPE

36



**Better
Quality**



**Saves
Money**



**Meet
Deadlines**

Prototyping Tool

37

- ▶ JustInMind
- ▶ www.justinmind.com

Sprint Guide Demo